```
"""
================================================================================
TELEMATICS CANN — SINGLE-FILE PIPELINE
Reproduction of the cross-sectional models of
Duval, Boucher & Pigeon (2024), ASTIN Bulletin 54(2), 239-262

Data: So, Boucher & Valdez (2021) synthetic driver telematics, 100,000 policies
      https://www2.math.uconn.edu/~valdez/telematics_syn-032021.csv

Scope: Poisson and Negative Binomial only. The MVNB specification is out of
       scope: this dataset is cross-sectional (one row per policy, no insured
       identifier), so there is no within-insured dependence for MVNB to model.

Run:   python cann_pipeline.py telematics_syn-032021.csv
       python cann_pipeline.py telematics_syn-032021.csv --seeds 5
       python cann_pipeline.py telematics_syn-032021.csv --skip-eda
================================================================================
"""

import argparse
import warnings

import numpy as np
import pandas as pd
import statsmodels.api as sm
import torch
import torch.nn as nn
from scipy.special import gammaln
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")
pd.set_option("display.width", 200)

# ============================================================================
# CONFIG
# ============================================================================
TRAD_NUM = ["Insured.age", "Car.age", "Credit.score", "Annual.miles.drive", "Year
TRAD_CAT = ["Insured.sex", "Marital", "Car.use", "Region"]
RESP = ["NB_Claim", "AMT_Claim"]
DROP = ["Duration", "Territory", "expo"]          # Territory: ~55 levels, exclud

# hand-picked telematics for the GLM benchmark (the paper's "engineered features"
TELE_GLM = ["Total.miles.driven", "Pct.drive.rush pm", "Pct.drive.wkend",
            "Avgdays.week", "Brake.14miles"]
```

```
HIDDEN = (64, 32, 16)
DROPOUT = 0.10
LR = 1e-3
BATCH = 1024
EPOCHS = 60
PATIENCE = 8


def banner(t):
    print("\n" + "=" * 78)
    print(t)
    print("=" * 78)


# ============================================================================
# LIKELIHOODS / METRICS  (numpy versions, for scoring)
# ============================================================================
def nll_poisson(y, mu):
    return float(-np.sum(y * np.log(mu) - mu - gammaln(y + 1)) / len(y))


def nll_nb(y, mu, alpha):
    r = 1.0 / alpha
    return float(-np.sum(gammaln(y + r) - gammaln(r) - gammaln(y + 1)
                         + r * np.log(r / (r + mu))
                         + y * np.log(mu / (r + mu))) / len(y))


def poisson_deviance(y, mu):
    """Comparable ACROSS distributions - use this, not raw NLL, to rank models."""
    t = np.where(y > 0, y * np.log(np.maximum(y, 1e-12) / mu), 0.0)
    return float(2 * np.mean(t - (y - mu)))


def gini(y, mu, expo):
    """Ordered Lorenz / concentration index on the predicted FREQUENCY score."""
    o = np.argsort(mu / expo)
    yy = y[o]
    c = np.cumsum(yy) / yy.sum()
    trap = getattr(np, "trapezoid", None) or np.trapz
    return float(1 - 2 * trap(c, dx=1.0 / len(yy)))


def lift_table(y, mu, expo, k=10):
    d = pd.DataFrame({"y": y, "mu": mu, "expo": expo})
    d["risk"] = d.mu / d.expo               # rank on FREQUENCY, not expected cou
```

```
    d["dec"] = pd.qcut(d.risk, k, labels=False, duplicates="drop")
    g = d.groupby("dec").apply(lambda x: pd.Series({
        "n": len(x),
        "exposure": x.expo.sum(),
        "observed_freq": x.y.sum() / x.expo.sum(),
        "predicted_freq": x.mu.sum() / x.expo.sum(),
    }))
    return g


def score(y, mu, expo, alpha=None):
    out = {
        "poisson_deviance": poisson_deviance(y, mu),
        "gini": gini(y, mu, expo),
        "balance": float(mu.sum() / y.sum()),      # 1.00 = unbiased in aggregate
        "own_nll": nll_nb(y, mu, alpha) if alpha else nll_poisson(y, mu),
    }
    return out


# ============================================================================
# 1. LOAD
# ============================================================================
def load(path):
    banner("1. LOAD & STRUCTURE")
    df = pd.read_csv(path)
    df["expo"] = df["Duration"] / 365.0          # Duration is in DAYS
    tele_all = [c for c in df.columns if c not in TRAD_NUM + TRAD_CAT + RESP + DR

    print(f"rows={len(df):,}  cols={df.shape[1]}   "
          f"(expect 100,000 x 52 for the published file)")
    print(f"traditional used = {len(TRAD_NUM)+len(TRAD_CAT)} "
          f"({len(TRAD_NUM)} numeric + {len(TRAD_CAT)} categorical); "
          f"Duration -> offset, Territory excluded")
    print(f"telematics = {len(tele_all)}   response = {len(RESP)}")
    print(f"missing={int(df.isna().sum().sum())}  duplicates={int(df.duplicated()
    print(f"exposure(yrs): min={df.expo.min():.3f} med={df.expo.median():.3f} "
          f"max={df.expo.max():.3f} total={df.expo.sum():,.0f}")
    print(f"policies with >=1 claim: {(df.NB_Claim>0).sum():,} "
          f"({(df.NB_Claim>0).mean():.3%})   [expect ~2,698 / ~2.7%]")
    return df, tele_all


# ============================================================================
# 2. EDA + DISPERSION DIAGNOSTICS
# ============================================================================
def eda(df, tele_all):
```

```
    banner("2A. CLAIM COUNT DISTRIBUTION")
    c = df.NB_Claim.value_counts().sort_index()
    print(pd.DataFrame({"n": c, "share": c / len(df)}).to_string())

    banner("2B. DISPERSION — UNCONDITIONAL")
    y = df.NB_Claim
    m, v = y.mean(), y.var(ddof=1)
    print(f"mean = {m:.6f}")
    print(f"var  = {v:.6f}")
    print(f"var/mean = {v/m:.4f}      (=1 Poisson-consistent, >1 overdispersed)")

    exp_zero = np.exp(-m) * len(df)
    print(f"\nzeros observed = {(y==0).sum():,}   Poisson({m:.4f}) predicts = {ex
          f"   excess = {(y==0).sum()-exp_zero:,.0f}")

    z = ((y - m) ** 2 - y) / (m * np.sqrt(2))
    t = z.mean() / (z.std(ddof=1) / np.sqrt(len(z)))
    print(f"\nCameron-Trivedi score test: t = {t:.2f}   (|t|>1.96 => reject equid

    banner("2C. ONE-WAY EXPOSURE-WEIGHTED FREQUENCY")
    for col in ["Insured.age", "Car.use", "Region", "Total.miles.driven", "Pct.dr
        if col not in df.columns:
            continue
        g = pd.qcut(df[col], 8, duplicates="drop") if df[col].dtype.kind in "if"
        t2 = df.groupby(g, observed=True).apply(lambda d: pd.Series({
            "n": len(d), "exposure": d.expo.sum(), "claims": d.NB_Claim.sum(),
            "freq": d.NB_Claim.sum() / d.expo.sum()}))
        print(f"\n--- {col} ---")
        print(t2.round(4).to_string())

    banner("2D. TELEMATICS COLLINEARITY (why telematics go in the MLP, not the GL
    corr = df[tele_all].select_dtypes(include=[np.number]).corr().abs()
    corr = corr.where(~np.eye(len(corr), dtype=bool), 0.0)
    top = corr.unstack().sort_values(ascending=False).drop_duplicates().head(12)
    print(top.round(3).to_string())
    print("\n-> pairs above ~0.9 are effectively one variable. A GLM cannot absor


# ============================================================================
# 3. SPLIT + ENCODE  (fit encoders on TRAIN only)
# ============================================================================
class Encoder:
    def __init__(self, tr, tele_all):
        self.tele_all = tele_all
        self.ref = pd.get_dummies(tr[TRAD_NUM + TRAD_CAT],
                                  columns=TRAD_CAT, drop_first=True).columns
        self.m, self.s = tr[TRAD_NUM].mean(), tr[TRAD_NUM].std().replace(0, 1)
```

```
        self.mt, self.st = tr[tele_all].mean(), tr[tele_all].std().replace(0, 1)

    def trad(self, d):
        X = pd.get_dummies(d[TRAD_NUM + TRAD_CAT], columns=TRAD_CAT, drop_first=T
        X = X.reindex(columns=self.ref, fill_value=0).astype(float)
        X[TRAD_NUM] = (d[TRAD_NUM].values - self.m.values) / self.s.values
        return X

    def tele(self, d, cols=None):
        cols = cols or self.tele_all
        return (d[cols] - self.mt[cols]) / self.st[cols]


def split(df, seed=42):
    banner("3. SPLIT 60/20/20 (stratified on claim occurrence)")
    strat = (df.NB_Claim > 0).astype(int)
    tr, tmp = train_test_split(df, test_size=0.40, random_state=seed, stratify=st
    va, te = train_test_split(tmp, test_size=0.50, random_state=seed,
                              stratify=(tmp.NB_Claim > 0).astype(int))
    for nm, d in [("train", tr), ("valid", va), ("test", te)]:
        print(f"{nm:6s} n={len(d):>7,}  exposure={d.expo.sum():>9,.0f}  "
              f"claims={d.NB_Claim.sum():>6,}  freq={d.NB_Claim.sum()/d.expo.sum(
    return tr.reset_index(drop=True), va.reset_index(drop=True), te.reset_index(d


# ============================================================================
# 4. GLM BASELINES
# ============================================================================
def fit_glms(tr, te, enc):
    banner("4. GLM BENCHMARKS")
    results, fitted = {}, {}

    for label, tele_cols in [("GLM-A traditional", None),
                             ("GLM-B trad+telematics", TELE_GLM)]:
        Xtr = enc.trad(tr)
        Xte = enc.trad(te)
        if tele_cols:
            cols = [c for c in tele_cols if c in tr.columns]
            Xtr = pd.concat([Xtr.reset_index(drop=True),
                             enc.tele(tr, cols).reset_index(drop=True)], axis=1)
            Xte = pd.concat([Xte.reset_index(drop=True),
                             enc.tele(te, cols).reset_index(drop=True)], axis=1)
        Xtr = sm.add_constant(Xtr, has_constant="add").astype(float)
        Xte = sm.add_constant(Xte, has_constant="add").astype(float)

        ytr, yte = tr.NB_Claim.values, te.NB_Claim.values
        otr, ote = np.log(tr.expo.values), np.log(te.expo.values)
```

```
    # --- Poisson
    pois = sm.GLM(ytr, Xtr, family=sm.families.Poisson(), offset=otr).fit(max
    mu_p = pois.predict(Xte, offset=ote)
    phi = float(np.sum((ytr - pois.mu) ** 2 / pois.mu) / (len(ytr) - Xtr.shap

    # --- Negative Binomial (profile alpha)
    best = None
    for a in np.linspace(0.05, 6.0, 40):
        try:
            m = sm.GLM(ytr, Xtr, family=sm.families.NegativeBinomial(alpha=a)
                       offset=otr).fit(maxiter=200)
            if best is None or m.llf > best[1]:
                best = (a, m.llf, m)
        except Exception:
            continue
    alpha, _, nb = best
    mu_n = nb.predict(Xte, offset=ote)

    print(f"\n--- {label}  ({Xtr.shape[1]} columns) ---")
    print(f"Poisson : AIC={pois.aic:>12,.1f}")
    print(f"NegBin  : AIC={nb.aic:>12,.1f}   alpha_hat={alpha:.4f}")
    print(f"Pearson dispersion phi = {phi:.4f}   "
          f"[>1 => residual overdispersion => NB justified]")
    print(f"LR (NB vs Poisson) = {2*(nb.llf-pois.llf):,.1f}   "
          f"[boundary test: halve the chi2_1 p-value]")

    s = pd.DataFrame({"coef": pois.params, "z": pois.tvalues})
    print("\ntop-10 |z|:")
    print(s.reindex(s.z.abs().sort_values(ascending=False).index).head(10).ro

    results[f"{label} | Poisson"] = score(yte, mu_p, te.expo.values)
    results[f"{label} | NegBin"] = score(yte, mu_n, te.expo.values, alpha=alp
    results[f"{label} | NegBin"]["alpha_hat"] = alpha
    results[f"{label} | Poisson"]["pearson_phi"] = phi
    fitted[label] = dict(pois=pois, nb=nb, alpha=alpha, Xcols=Xtr.columns)

banner("4B. DOES TELEMATICS REDUCE UNEXPLAINED HETEROGENEITY?")
a0 = results["GLM-A traditional | NegBin"]["alpha_hat"]
a1 = results["GLM-B trad+telematics | NegBin"]["alpha_hat"]
print(f"alpha_hat traditional only     = {a0:.4f}")
print(f"alpha_hat traditional+telematics = {a1:.4f}")
print(f"change = {100*(a1-a0)/a0:+.1f}%")
print("A DROP means telematics explained part of the frailty the rating facto
print("This is one of the most persuasive single results in the project.")
return results, fitted
```

```
# ============================================================================
# 5. CANN
# ============================================================================
class CANN(nn.Module):
    """
    log mu = log(expo) + x_trad . beta            <- log-linear (actuarial) part
                       + MLP(x_trad, x_tele)      <- network (boosting) part

    beta initialised at the Poisson GLM solution; MLP output layer initialised at
    ZERO, so at epoch 0 the CANN IS the GLM and can only move away from it by
    improving the likelihood. That is what makes this a CANN and not a plain MLP.
    """

    def __init__(self, p_trad, p_tele, beta0, hidden=HIDDEN, dropout=DROPOUT):
        super().__init__()
        self.lin = nn.Linear(p_trad, 1)
        with torch.no_grad():
            self.lin.weight.copy_(torch.tensor(beta0[1:], dtype=torch.float32).vi
            self.lin.bias.copy_(torch.tensor(beta0[:1], dtype=torch.float32))

        layers, d = [], p_trad + p_tele
        for h in hidden:
            layers += [nn.Linear(d, h), nn.BatchNorm1d(h), nn.Tanh(), nn.Dropout(
            d = h
        out = nn.Linear(d, 1)
        nn.init.zeros_(out.weight)
        nn.init.zeros_(out.bias)
        self.mlp = nn.Sequential(*layers, out)

        # NB dispersion: ONE global trainable scalar, on the log scale so r > 0.
        # It is NOT an output of the forward pass - Adam updates it like any weig
        self.log_r = nn.Parameter(torch.tensor(0.0))

    def forward(self, xt, xl, off):
        eta = off + self.lin(xt).squeeze(-1) + self.mlp(torch.cat([xt, xl], 1)).s
        return torch.exp(torch.clamp(eta, -12, 6))


def t_poisson_nll(y, mu):
    return (mu - y * torch.log(mu + 1e-10)).mean()


def t_nb_nll(y, mu, log_r):
    r = torch.exp(log_r)
    return -(torch.lgamma(y + r) - torch.lgamma(r) - torch.lgamma(y + 1)
             + r * torch.log(r / (r + mu))
```

```
             + y * torch.log(mu / (r + mu) + 1e-10)).mean()


def pack(d, enc):
    return (torch.tensor(enc.trad(d).values.astype(np.float32)),
            torch.tensor(enc.tele(d).values.astype(np.float32)),
            torch.tensor(d.NB_Claim.values.astype(np.float32)),
            torch.tensor(np.log(d.expo.values).astype(np.float32)))


def train_cann(dist, packs, beta0, seed=0, verbose=True):
    torch.manual_seed(seed)
    (Xtr, Ttr, ytr, otr), (Xva, Tva, yva, ova), _ = packs
    net = CANN(Xtr.shape[1], Ttr.shape[1], beta0)
    opt = torch.optim.Adam(net.parameters(), lr=LR)
    n, best, state, wait = len(ytr), np.inf, None, 0

    for ep in range(EPOCHS):
        net.train()
        perm = torch.randperm(n)
        for i in range(0, n, BATCH):
            j = perm[i:i + BATCH]
            if len(j) < 2:
                continue
            mu = net(Xtr[j], Ttr[j], otr[j])
            loss = t_poisson_nll(ytr[j], mu) if dist == "poisson" \
                else t_nb_nll(ytr[j], mu, net.log_r)
            opt.zero_grad()
            loss.backward()
            nn.utils.clip_grad_norm_(net.parameters(), 5.0)
            opt.step()

        net.eval()
        with torch.no_grad():
            muv = net(Xva, Tva, ova)
            v = (t_poisson_nll(yva, muv) if dist == "poisson"
                 else t_nb_nll(yva, muv, net.log_r)).item()
        if v < best - 1e-6:
            best, state, wait = v, {k: p.clone() for k, p in net.state_dict().ite
        else:
            wait += 1
            if wait >= PATIENCE:
                if verbose:
                    print(f"    early stop @ epoch {ep}")
                break
        if verbose and ep % 10 == 0:
            print(f"    epoch {ep:3d}  valid NLL = {v:.6f}")
```

```
    net.load_state_dict(state)
    return net, best


def run_cann(tr, va, te, enc, fitted, seeds=1):
    banner("5. CANN")
    packs = (pack(tr, enc), pack(va, enc), pack(te, enc))
    (Xte, Tte, yte, ote) = packs[2]

    # warm start from the traditional-only Poisson GLM
    Xg = sm.add_constant(enc.trad(tr), has_constant="add").astype(float)
    glm = sm.GLM(tr.NB_Claim.values, Xg, family=sm.families.Poisson(),
                 offset=np.log(tr.expo.values)).fit(maxiter=200)
    beta0 = glm.params.values.astype(np.float32)
    print(f"warm start from Poisson GLM: intercept={beta0[0]:.4f}, {len(beta0)-1}
    print(f"MLP inputs: {Xte.shape[1]} traditional + {Tte.shape[1]} telematics")

    results, seed_log = {}, []
    for dist in ["poisson", "nb"]:
        print(f"\n--- CANN [{dist}] ---")
        runs = []
        for s in range(seeds):
            net, bv = train_cann(dist, packs, beta0, seed=s, verbose=(s == 0))
            net.eval()
            with torch.no_grad():
                mu = net(Xte, Tte, ote).numpy()
                r_hat = float(torch.exp(net.log_r))
            alpha = 1 / r_hat if dist == "nb" else None
            sc = score(yte.numpy(), mu, te.expo.values, alpha=alpha)
            if dist == "nb":
                sc["alpha_hat"] = alpha
            runs.append(sc)
            seed_log.append(dict(model=f"CANN-{dist}", seed=s, **sc))
            print(f"  seed {s}: dev={sc['poisson_deviance']:.6f} gini={sc['gini']
                  + (f" alpha={alpha:.4f}" if alpha else ""))

        agg = {k: float(np.mean([r[k] for r in runs])) for k in runs[0]}
        if seeds > 1:
            agg["dev_sd"] = float(np.std([r["poisson_deviance"] for r in runs]))
        results[f"CANN | {'Poisson' if dist=='poisson' else 'NegBin'}"] = agg

    if seeds > 1:
        banner("5B. SEED STABILITY")
        print(pd.DataFrame(seed_log).round(6).to_string(index=False))
        print("\nA single-seed improvement over the GLM is not evidence. Report m
```

```
    # keep the last NB model's predictions for the lift table
    return results, mu, yte.numpy()


# ============================================================================
# 6. COMPARISON
# ============================================================================
def compare(glm_res, cann_res, mu_nb, y_te, expo_te):
    banner("6. MODEL COMPARISON")
    allr = {**glm_res, **cann_res}
    tab = pd.DataFrame(allr).T
    order = ["poisson_deviance", "gini", "balance", "own_nll", "alpha_hat", "pear
    tab = tab[[c for c in order if c in tab.columns]]
    print(tab.round(6).to_string())

    print("\nHOW TO READ THIS TABLE")
    print("  poisson_deviance : PRIMARY ranking metric. Comparable across all fou
    print("  gini             : discrimination / risk ranking. Higher = better.")
    print("  balance          : sum(pred)/sum(obs). Must be ~1.00 or the model is
    print("                     for pricing regardless of its deviance.")
    print("  own_nll          : each model's own likelihood. NOT comparable betwe
    print("                     Poisson and NB - they are likelihoods of differen
    print("  alpha_hat        : residual unexplained heterogeneity. Lower = the c
    print("                     explained more of the frailty.")

    base = tab.loc["GLM-A traditional | Poisson", "poisson_deviance"]
    print("\nIMPROVEMENT IN POISSON DEVIANCE vs GLM-A (traditional, Poisson):")
    for k in tab.index:
        print(f"  {k:38s} {100*(base - tab.loc[k,'poisson_deviance'])/base:+7.2f}

    print("\nTHE HEADLINE TEST (the paper's claim):")
    print("  Does 'CANN | NegBin' beat 'GLM-B trad+telematics | NegBin'?")
    print("  i.e. does a learned telematics representation beat hand-engineered f
    a = tab.loc["CANN | NegBin", "poisson_deviance"]
    b = tab.loc["GLM-B trad+telematics | NegBin", "poisson_deviance"]
    print(f"  CANN-NB deviance = {a:.6f}   GLM-B-NB deviance = {b:.6f}")
    print(f"  => {'CANN WINS' if a < b else 'GLM WINS'} by {abs(100*(b-a)/b):.2f}

    banner("6B. LIFT / DECILE CALIBRATION (CANN-NB)")
    print(lift_table(y_te, mu_nb, expo_te).round(5).to_string())
    print("\nobserved_freq should rise monotonically across deciles (discriminati
    print("and track predicted_freq closely within each decile (calibration).")
    return tab


# ============================================================================
# MAIN
```

```
# ============================================================================
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("path", nargs="?", default="telematics_syn-032021.csv")
    ap.add_argument("--seeds", type=int, default=1)
    ap.add_argument("--skip-eda", action="store_true")
    ap.add_argument("--out", default="model_comparison.csv")
    a = ap.parse_args()

    df, tele_all = load(a.path)
    if not a.skip_eda:
        eda(df, tele_all)

    tr, va, te = split(df)
    enc = Encoder(tr, tele_all)

    glm_res, fitted = fit_glms(tr, te, enc)
    cann_res, mu_nb, y_te = run_cann(tr, va, te, enc, fitted, seeds=a.seeds)
    tab = compare(glm_res, cann_res, mu_nb, y_te, te.expo.values)

    tab.to_csv(a.out)
    print(f"\nsaved -> {a.out}")


if __name__ == "__main__":
    main()
```